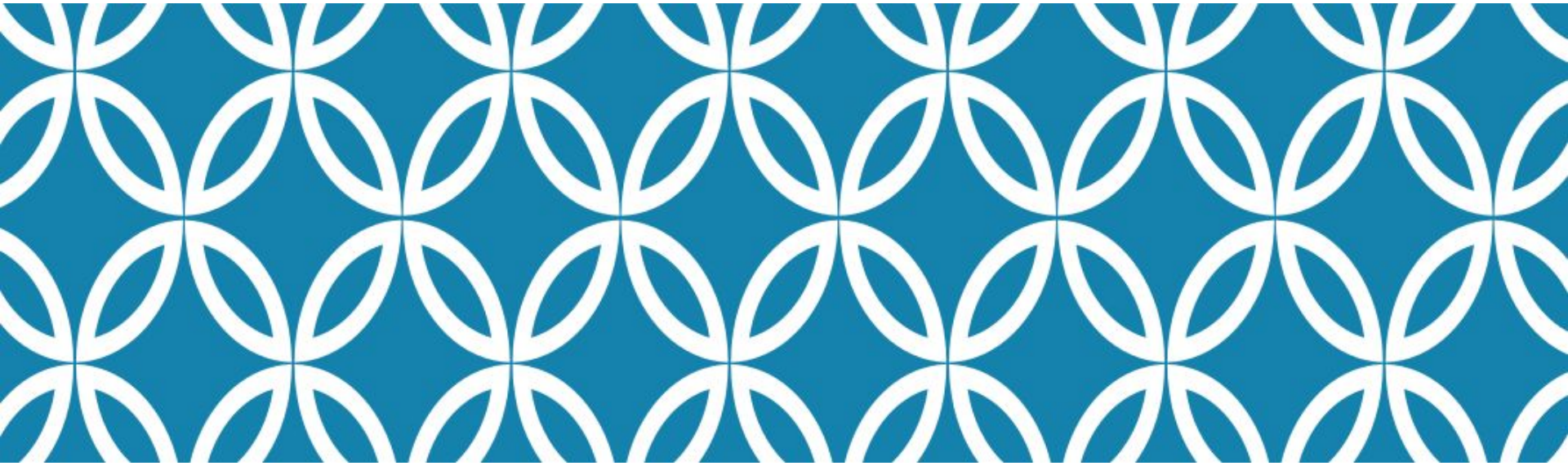
MABLAB

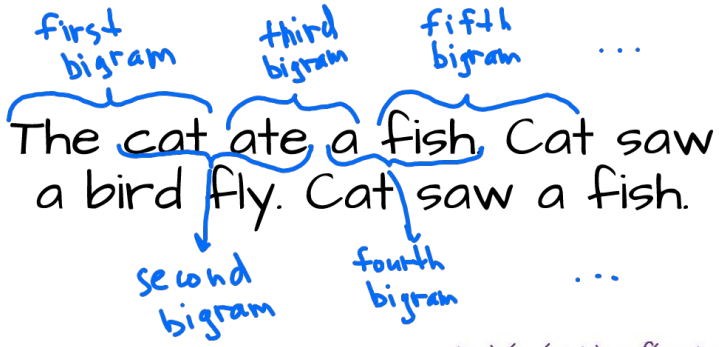


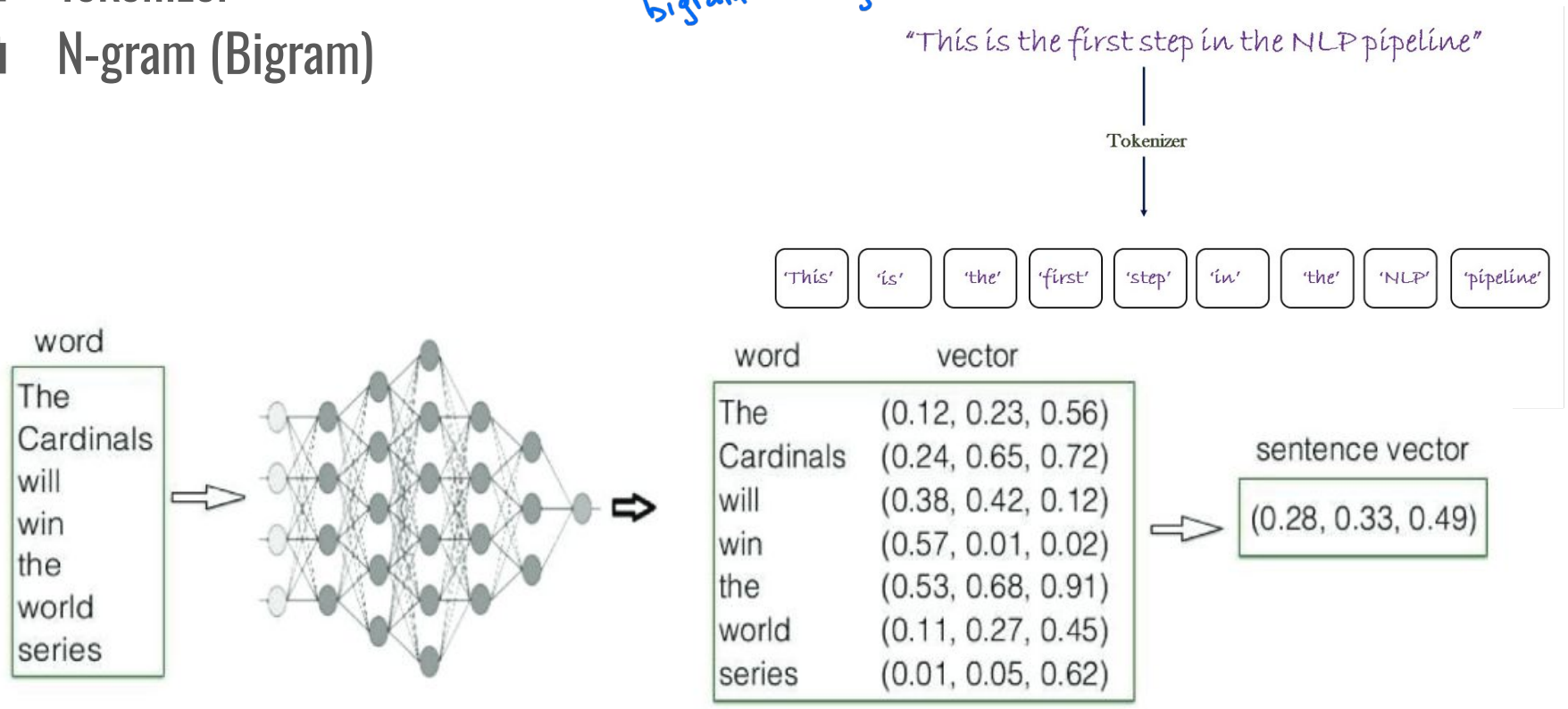
PROJECT
Next Word Prediction

Prepared by Mr. Nop Phearum

Content

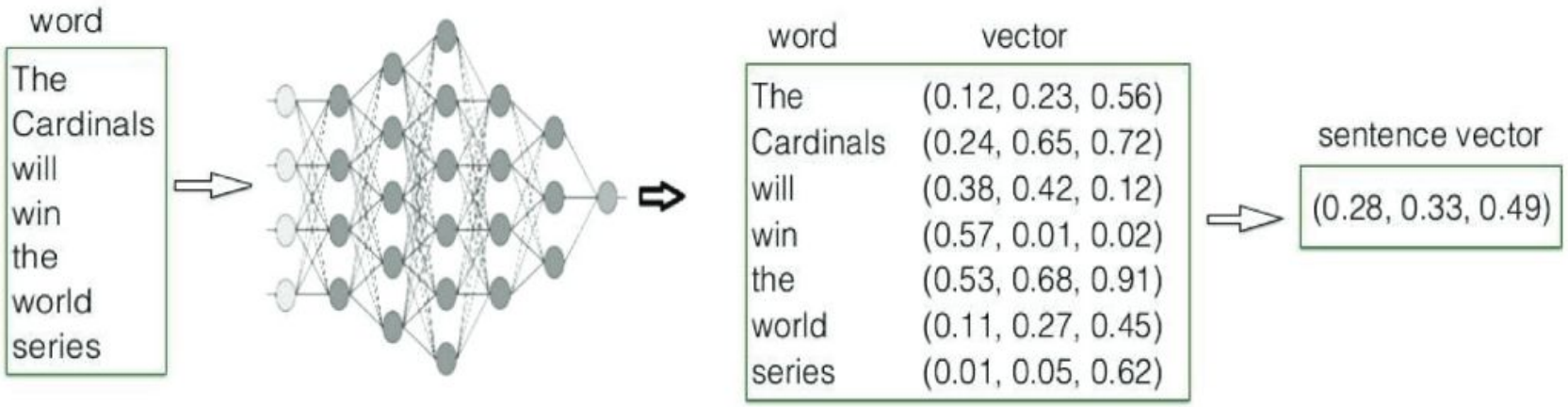
- Text to Vector
- Tokenizer
- N-gram (Bigram)





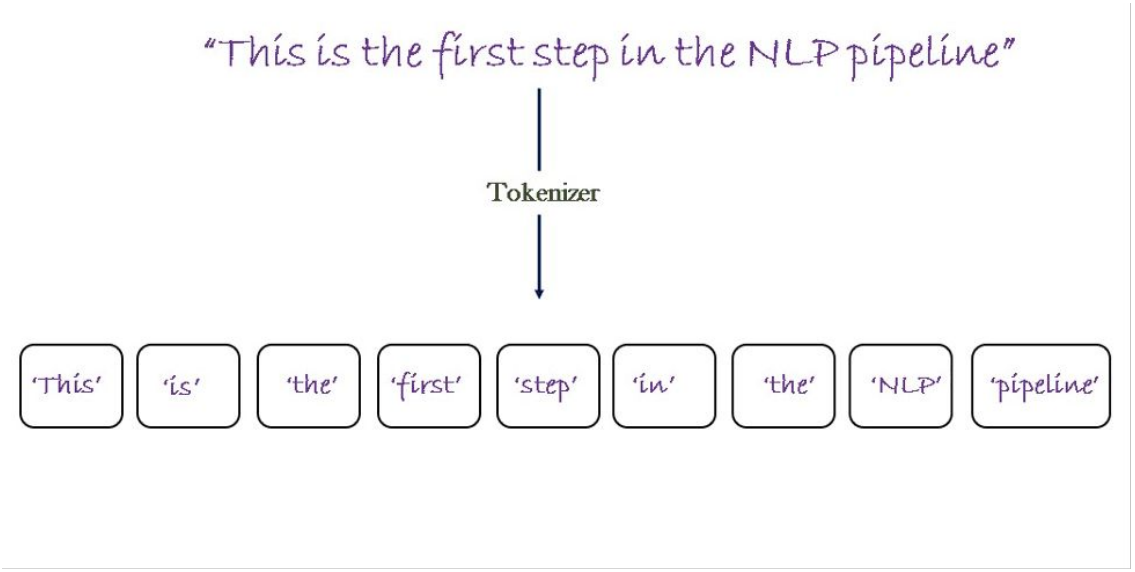
Text to Vector

This method converts text into numerical vector representations that capture semantic meaning. Models like **Word2Vec**, **GloVe**, or **FastText** map words to dense vector spaces where similar words cluster together. These vectors can be used to predict the next word by finding words with vectors close to the context's vector. Modern approaches like **BERT** and transformer models use contextual embeddings where a word's vector changes based on surrounding context.



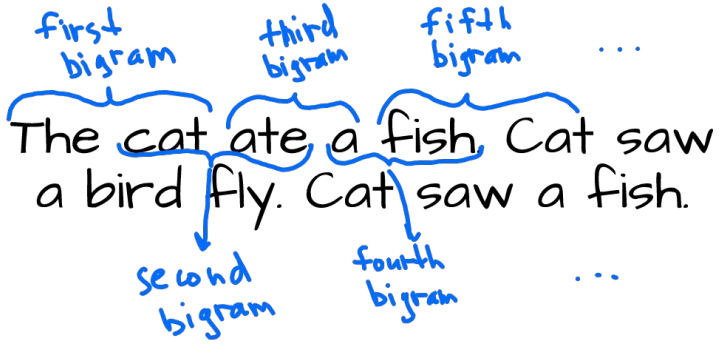
Tokenizer

Tokenizers break text into smaller units (tokens) like **words**, **subwords**, or **characters**. For word prediction, tokenizers first process input text, then the model assigns probabilities to potential next tokens. Modern tokenizers like WordPiece, BPE, or SentencePiece use subword units to handle vocabulary more efficiently. The tokenization strategy greatly affects prediction quality, especially for rare words or morphologically rich languages.



N-gram (Bigram)

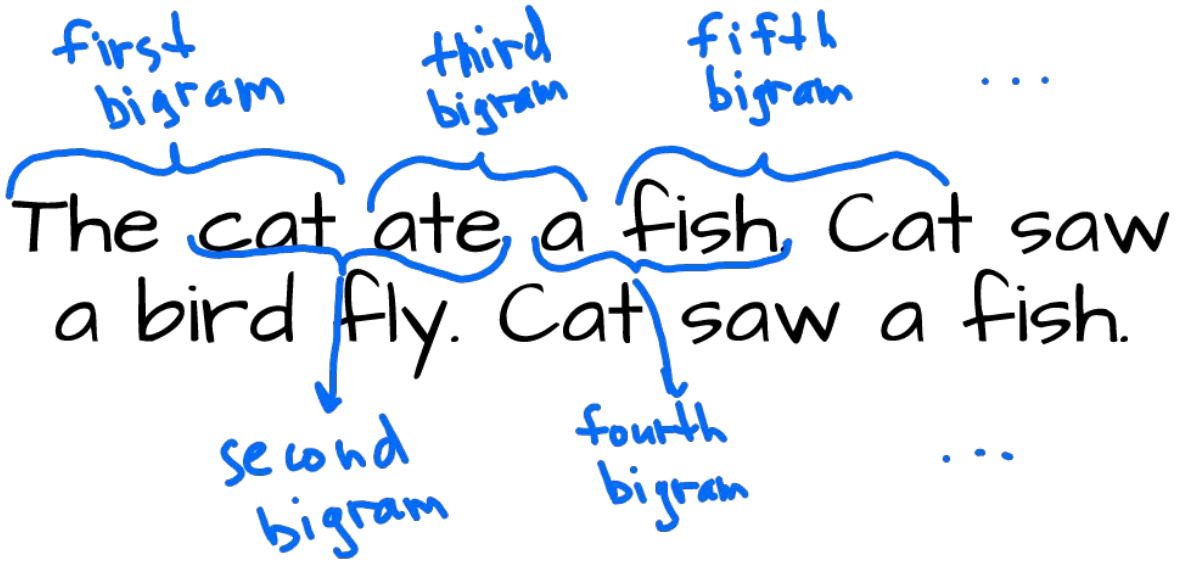
N-grams are sequences of n consecutive tokens. Bigrams specifically consider pairs of adjacent words. For prediction, bigram models calculate the probability of a word appearing after another based on training data frequencies. While simpler than neural approaches, n-gram models can be effective for specific applications. They work on the Markov assumption that the next word depends only on the previous **n-1** words.



Unigrams	"patient"	"has"	"evidence"	"of"	"macular"	"degeneration"
Bigrams	"patient has"	"has evidence"	"evidence of"	"of macular"	"macular degeneration"	
Trigrams	"patient has evidence"	"has evidence of"	"evidence of macular"	"of macular degeneration"		
4-grams	"patient has evidence of"	"has evidence of macular"	"evidence of macular degeneration"			

Project Set Up | Project Overview

In this assignment, you will build a text prediction model using MATLAB that can predict the next word in a sequence. This project will introduce you to fundamental concepts in natural language processing and give you hands-on experience with statistical language modeling.



Learning Objectives

By completing this project, you will:

- Implement text preprocessing and tokenization techniques
- Build a statistical n-gram language model
- Develop vector representations for words
- Create a functional word prediction system
- Evaluate the performance of different prediction approaches

Project Requirements

Part 1: Data Preparation (15%)

1. Choose a text corpus (options provided below or select your own with approval)
2. Implement text preprocessing functions:
   - Convert text to lowercase
   - Remove punctuation and special characters
   - Split text into tokens (words)
   - Create a vocabulary of unique words
3. Analyze and visualize basic statistics of your corpus:
   - Vocabulary size
   - Word frequency distribution
   - Most common words

Project Requirements

Part 2: N-gram Model Implementation (30%)

1. Build a bigram model:
   - Create a frequency table of word pairs
   - Calculate transition probabilities
   - Implement a function that predicts the most likely next word
2. Extend to a trigram model (optional for extra credit)
3. Implement at least one smoothing technique to handle unseen word combinations

Project Requirements

Part 3: Vector Representation (25%)

1. Create vector representations for words:
   - Implement one-hot encoding as a baseline
   - Create simple co-occurrence based word embeddings
2. Develop a prediction function that uses vector similarity
3. Compare the predictions from your vector approach with the n-gram approach

Project Requirements

Part 4: Model Evaluation (15%)

1. Split your corpus into training and testing sets
2. Implement functions to evaluate your model:
   - Calculate prediction accuracy
   - Measure perplexity (optional)
   - Compare performance of different approaches
3. Analyze and document the strengths and weaknesses of each approach

Project Requirements

Part 5: Application and Documentation (15%)

1. Create a simple UI for demonstrating your word prediction model
2. Save and load functionality for your trained model
3. Comprehensive documentation including:
   - Clear code comments
   - Function descriptions
   - Results analysis
   - Suggestions for improvement

Project Requirements

- **Week 1:**
    - Data preparation and exploration
    - N-gram model implementation
- **Week 2-3:**
    - Vector representations
    - Evaluation and application
    - Documentation and presentation